Firmware Fuzzing: The State of the Art

Chi Zhang
zhangchi_seg@smail.nju.edu.cn
State Key Laboratory of Novel Software Technology
Nanjing University
Nanjing, Jiangsu Province, China

Yu Wang
yuwang_cs@smail.nju.edu.cn
State Key Laboratory of Novel Software Technology
Nanjing University
Nanjing, Jiangsu Province, China

Linzhang Wang
lzwang@nju.edu.cn
State Key Laboratory of Novel Software Technology
Nanjing University
Nanjing, Jiangsu Province, China

ABSTRACT

Background: Firmware is the enable software of Internet of Things (IoT) devices, and its software vulnerabilities are one of the primary reason of IoT devices being exploited. Due to the limited resources of IoT devices, it is impractical to deploy sophisticated run-time protection techniques. Under an insecure network environment, when a firmware is exploited, it may lead to denial of service, information disclosure, elevation of privilege, or even life-threatening. Therefore, firmware vulnerability detection by fuzzing has become the key to ensure the security of IoT devices, and has also become a hot topic in academic and industrial research. With the rapid growth of the existing IoT devices, the size and complexity of firmware, the variety of firmware types, and the firmware defects, existing IoT firmware fuzzing methods face challenges. **Objective:** This paper summarizes the typical types of IoT firmware fuzzing methods, analyzes the contribution of these works, and summarizes the shortcomings of existing fuzzing methods. **Method:** We design several research questions, extract keywords from the research questions, then use the keywords to search for related literature. **Result:** We divide the existing firmware fuzzing work into real-device-based fuzzing and simulation-based fuzzing according to the firmware execution environment, and simulation-based fuzzing is the mainstream in the future; we found that the main types of vulnerabilities targeted by existing fuzzing methods are memory corruption vulnerabilities; firmware fuzzing faces more difficulties than ordinary software fuzzing. **Conclusion:** Through the analysis of the advantages and disadvantages of different methods, this review provides guidance for further improving the performance of fuzzing techniques, and proposes several recommendations from the findings of this review.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging**; • **Computer systems organization** → **Firmware**.

KEYWORDS

Internet of Things, firmware, fuzzing, literature review

Internetware'20, May 12–14, 2021, Singapore, Singapore

ACM ISBN 978-1-4503-8819-1/20/11…$15.00
https://doi.org/10.1145/3457913.3457934

ACM Reference Format:
Chi Zhang, Yu Wang, and Linzhang Wang. 2020. Firmware Fuzzing: The State of the Art. In *12th Asia-Pacific Symposium on Internetware (Internetware'20), May 12–14, 2021, Singapore, Singapore.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3457913.3457934

1 INTRODUCTION

In recent years, the improvement of hardware computing power, the acceleration of communication speed, and the increase of communication methods give more application scenarios for Internet of Things (IoT) devices. Currently, the number of IoT devices has maintained a rapid growth trend, but the number of attacks against IoT devices had also increased even faster. For example, criminals used IoT devices to create the largest denial of service attack on Github in history [25]. Therefore, both academia and industry have paid more and more attention to detecting vulnerabilities in the firmware of IoT, and IoT devices have become a hot spot for many researchers. The firmware in the IoT device refers to all the software in the IoT device. There are many methods for testing firmware, such as static analysis [11], symbolic execution [14], fuzzing [57], machine-learning-based vulnerabilities prediction [54], etc., but fuzzing is one of the most studied methods.

Fuzzing refers to the method that provides a large number of normal and abnormal test cases to the program and monitors the states of the program to detect exceptions [37]. There are two main categories for generating test cases: generation-based methods and mutation-based methods. The generation-based methods generate test cases that meet the specifications by modeling the software's input format, run-time environment, and communication protocols. There are many works in this category, among which well-known ones are Peach [18], Sulley [2], MoWF [42], and so on. The mutation-based methods generate new test cases by applying mutation operations to existing valid test cases. Classic works such as AFL [27], Honggfuzz [49], BuzzFuzz [22],etc. Fuzzing has yielded promising results in application-level software testing, thus this method has been extended to many areas such as kernel testing (e.g., Syzkaller [26]), firmware testing (e.g., FIRM-AFL [57]), protocol fuzzing(e.g., Autofuzz [28]),etc.

The organization of this article is as follows: The research methodology used in this review is discussed in detail in Section 2. Then, the review results that answer the research questions are presented in Section 3. Section 4 discusses the deficiencies of firmware fuzzing and future improvements. Lastly, section 5 concludes the review.

2 RESEARCH METHOD

This literature review, which follows a systematic approach, was done by adapting the guidelines provided by Keele et al. [33] for

performing a systematic review. There are many activities in the guidelines, the main ones include identification of the need for a review, specifying the research questions, selection of primary studies, formatting the main report, etc.

2.1 Research Questions

In order to understand the state of the firmware fuzzing, we propose the research questions shown below:

- **RQ1:** What is the execution environment of firmware fuzzing?
- **RQ2:** What are the main types of vulnerabilities that firmware fuzzing can detect?
- **RQ3:** What is the difference between firmware fuzzing and general software fuzzing?

2.2 Study Selection

We select electronic sources that have been widely used in existing systematic reviews [31]. The chosen sources are listed below:

(1) IEEExplore[1]
(2) ACM Digital Library[2]
(3) SpringerLink[3]
(4) Google Scholar[4]

The search keywords including "firmware fuzzing", "IoT security", "firmware simulation", and so on. These keywords are related to the research question we proposed. We focus on Internet of Things security and have collected a lot of papers. But the Internet of Things includes cloud platforms, smart terminals, Internet of Things devices, etc. We only focus on the firmware in the Internet of Things devices. There are many testing technologies for IoT firmware, and their targets are not only the complete firmware. Testing techniques include static analysis [11, 13, 29, 44, 45], symbolic execution [5, 14, 17, 30, 48], fuzzing [4, 7, 9, 10, 12, 19, 34, 35, 39–41, 53, 56, 57], formal verification [8, 38, 50], and machine-learning-based vulnerabilities prediction [15, 16, 20, 21, 23, 24, 43, 54, 55, 58]. Targets include the complete firmware, firmware system, firmware communication protocol, firmware unpacking files, etc. We focus our attention on testing the complete firmware and collected 37 papers. We selected 12 papers on firmware fuzzing to answer our questions.

3 RESULT

After the data have been extracted from the selected primary studies, data synthesis was done. We have summarized the existing work to answer our questions.

3.1 Simulation-based fuzzing is the mainstream in the future. (RQ1)

Benefiting from the improvement of hardware computing power and the acceleration of communication speed, and the increase of communication techniques, the types and forms of IoT devices are growing rapidly. For example, the CPU architecture of IoT devices includes ARM, MIPS, X86, etc. On-chip peripherals include I2C, USART, GPIO, etc. Off-chip peripherals include wireless, Bluetooth, ZigBee, gyroscope, thermometer, etc. The rapid growth of CPU architecture types and peripheral types has brought great difficulties to the simulation of IoT devices. So the early firmware fuzzing was carried out on real device.

RPFuzzer [53] detects vulnerabilities in router protocols by sending normal packets to real router devices while monitoring routers' CPU utilization and checking system logs to detect DoS vulnerabilities and router reboot. IOTFuzzer [10] obtains the information in the communication protocol of the IoT firmware by analyzing the interaction between mobile applications with IoT firmware, and generates test cases based on this information. This process does not require obtaining the firmware of the IoT device. Asadollah et al. [4] automatically collect event information in the embedded operating system FreeRTOS [1] and determine whether there are concurrency vulnerabilities. But the focus of this work is on the embedded operating system.

Real-device-based fuzzing can obtain the most realistic execution information of the device, but there are still great constraints. Firstly, the available execution information is limited. Commercial IoT devices usually do not provide users with debugging interfaces, so specific run-time information cannot be obtained, such as execution paths. Secondly, in the real environment, many extreme cases are difficult to build, so it is difficult to improve the code coverage of fuzzing, which is one of the main indicators for evaluating the quality of fuzzing technology. Finally, the test needs to obtain a large number of real devices, and manually connect and debug them, which requires a lot of economic and time costs. Simulation-based fuzzing can solve the above problems and become the current research mainstream. The current mainstream virtual platform includes QEMU [6] and Simics [46]. QEMU has become a hotspot in academic research due to its open source and easy to use. However, due to the insufficient ability of the simulator to simulate peripherals, current research focuses on using different methods to make up for this deficiency.

Avatar [56] solves the problem that the simulator cannot simulate peripherals by executing code in the simulator and performing I/O operations in the real device. SURROGATES[35] is based on Avatar and improves the ability to communicate between the simulator and the hardware. Avatar2 [40] has more powerful orchestration capabilities that allowing different analysis frameworks, debugging tools, simulators, and real devices to interact with each other. FIRMADYNE [9] based on the full system emulation of QEMU and an instrumented kernel to achieve the ability that fuzzing the Linux-based firmware of network-connected commercial-off-the-shelf devices in the simulator. FIRM-AFL [57] based on FIRMADYNE and reconcile the full-system emulation and user-mode emulation of QEMU to improve simulator performance. FirmAE[34] allows FIRMADYNE to simulate more firmware. Laelaps [7] uses concolic execution to generate qualified peripheral inputs through constraint solving to make up for the simulator's inability to simulate peripherals and does not require any knowledge of firmware. It also models parts of the ARM Cortex-M based embedded microcontroller which is essential but full system emulation missing. P2IM [19] and DICE [39] model peripherals to generate peripheral inputs that keep the firmware running smoothly. HALucinator [12] decouples the hardware from the firmware by replacing the Hardware

[1] https://ieeexplore.ieee.org/
[2] https://dl.acm.org/
[3] https://link.springer.com/
[4] https://scholar.google.com/

Abstraction Layers (HAL) code, then the simulator can simulate peripherals without obstacles.

3.2 Memory corruption vulnerabilities are the primary detection targets. (RQ2)

Real-device-based fuzzing can only obtain limited information, so it is impossible to judge the exact type and specific location of the vulnerabilities. RPFuzzer [53] monitors the CPU utilization to detect DoS vulnerabilities, send normal messages to routers, and analyze system logs to detect system crashes, reboot and zombie process. IOTFuzzer [10] aims to find memory corruption vulnerabilities in IoT devices and infer the aliveness of the device by sending heartbeat messages. Asadollah et al. [4] uses the Tracealyzer tool for logging relevant events and infer whether there are concurrency vulnerabilities.

The purpose of Avatar [56], SURROGATES [35] and Avatar2 [40] as orchestration frameworks is to enable different tools to interact with each other, and it does not have the ability to detect vulnerabilities. FIRMADYNE [9] and FirmAE [34] implemented automated dynamic analysis passes which are registered as callbacks, to detect vulnerabilities include information leak, buffer overflow, and command injection, and so on. FIRM-AFL [57], Halucinator [12], P2IM [19] and DICE [39] use AFL [27] as fuzzer and determine whether there is a memory corruption vulnerability based on the crashes. Laelaps [7] based on the work of Muench et al. [41] and detect six types of memory corruption vulnerabilities using PANDA [36] plugin.

The information of vulnerabilities detected by these fuzzing tools is listed in Table 1 in Appendix A. Through comparison, we find that the run-time information that can be obtained by real-device-based fuzzing is limited, so that it is difficult to find the exact type and location of the vulnerabilities. Simulation-based fuzzing can obtain more detailed firmware run-time information such as execution path, instruction, and variable value, through pass or QEMU/PANDA plug-in. The situation is concretely reflected in the work FIRMADYNE [9] and Laelaps [7]. At this stage, the work is still focused on making up for the lack of simulators in simulating hardware. Therefore, many works does not focus on finding vulnerabilities, but uses another important dimension in fuzzing-code coverage to indicate their ability to generate input from peripherals.

3.3 Firmware fuzzing faces more difficulties than general software fuzzing. (RQ3)

Because the execution environment of firmware is different from that of general software, there are some differences in the difficulties of fuzzing.

First, the difficulty of obtaining binary files is different. When we fuzz general software that we have permission to use, we can definitely get its binary files and even the source code. But in the firmware test, it is completely different. The firmware is stored in the embedded device, and is sold together with the device. The firmware upgrade can be fully automated without the need for additional downloads from the website. The user does not have enough tools and capabilities to extract the firmware from the device. Make bricks without straw, but IOTFuzzer [10] proposes a method to directly interact with the device without obtaining the firmware in the device, and it solves the communication encryption problem when the IoT device interacts with the mobile app.

Second, the execution environment of firmware is difficult to build. General software fuzzing obtains specific information at run-time by instrumentation at compile time, or through hardware assistance such as IntelPT [32]. Real-device-based fuzzing can run at a high enough speed but cannot obtain enough run-time information. Fortunately, there is a debugging component ETM [3] in the Arm Cortex-M microcontroller, which has a function similar to IntelPT, which can be used to trace the execution path of the firmware, but this function requires a specific physical interface and a debugging tool J-Trace [47]. Simulation-based fuzzing can obtain enough run-time information but cannot reach the speed of general software fuzzing. FIRM-AFL [57] combines full system emulation and user-mode emulation to improve the performance of simulation, but it is still incomparable with actual execution.

Third, firmware binary is more complicated than general software. As the underlying hardware architecture of IoT devices is richer, the firmware binary instruction set is more complex than general software. And the firmware needs to consider interacting with the underlying hardware, which is unnecessary for general software. These challenges have caused huge obstacles when integrating static analysis, symbolic execution, and other technologies. For example, symbolic execution needs to be adapted to a specific instruction set and memory architecture. Laelaps [7] is adapted to the symbolic execution of Arm Cortex-M series processors.

Fourth, firmware input is more complicated than general software. The peripheral input of the firmware can be divided into two categories. One is the peripheral input from the perception layer, such as microphone, camera, gravity sensor, temperature sensor, gyroscope, etc. This type of peripheral input is mainly based on images, sounds, and numerical values, and finally written into the memory in numerical form. The construction of this input requires domain knowledge of some specific peripherals, such as video encoding and decoding, etc. But the fuzzing of general software does not need to consider these, only information such as videos and pictures are provided as test cases to the software. The other is the communication peripheral input from the transmission layer, such as wireless, Bluetooth, ZigBee, etc. These peripheral inputs are mainly in the form of packets and need to be written into the memory in the form of strings, but this type of input often contains state, and the structure of the input needs to model the communication protocols and devices use environment. This type peripheral input is similar to the protocol test of general software. Another important input in the firmware is the interrupt. This input is relatively rare in general software. To determine its input time and input value, it is necessary to conduct an in-depth analysis of the code and use environment. Laelaps [7], P2IM [19] and DICE [39] all added support for interrupts in the fuzzing.

4 DISCUSSION

With the explosive growth of the number of IoT devices and the gradual application of IoT devices in important areas of production and life, the security of Internet of Things devices has become a research focus in academia and industry. Existing fuzzing testing tools

have been used in commercial Internet of Things devices. Many vulnerabilities have been found in the Internet of Things devices, but these tasks are a drop in the bucket relative to a large number of IoT devices. Current academic research is far from commercial applications. There are many reasons for this.

In the first place, the difficulty of obtaining firmware limits the researchers to work in the field. In this case, security researchers can only use limited debugging and testing methods, or use other technologies such as side-channel attacks to discover vulnerabilities in IoT devices, but these technologies are difficult to find vulnerabilities hidden in the firmware and cannot be effectively guaranteed firmware security. If firmware benchmarks are available for researchers in the future, they can save the time of researchers constructing data sets, provide unified test standards, and finally promote the development of firmware testing.

Secondly, the lack of simulators restricts the process of research. Through research, it is difficult to find more types of vulnerabilities in real-device-based fuzzing, and simulation-based fuzzing will be the mainstream in the future. The ability of the simulator is the main factor restricting the effect of simulation-based fuzzing, but existing simulators still have great limitations when simulating different hardware. Manually modifying the simulator to add support for other hardware is a difficult and time-consuming work, and affects the scalability of the tool. If the simulator can be easily compatible with new hardware in the future, it can enable researchers to focus on solving the problems of the test method itself.

Furthermore, the current simulation-based fuzzing does not make full use of the rich information obtained from the simulator to determine whether there are vulnerabilities. Many works still depend on checking the system status to determine whether there are vulnerabilities [12, 57]. On the contrary, SDRacer [51, 52] tries to leverage the virtual platform to detect bugs while monitoring run-time information of systems. Moreover, Laelaps [7] has proven that the PANDA plug-in is sufficient to detect more vulnerabilities. These techniques, as a promising way of improving the effectiveness of fuzzing, can be used to detect more complicated and critical software vulnerabilities. In the future, vulnerability detection can be done as a separate work, integrated into the emulation platform in the form of plug-in, to improve the ability of fuzzing tools to detect vulnerabilities. Besides, future work can use vulnerability detection capabilities as a major standard to reflect the quality of the capabilities, because in many cases IoT devices will not crash when encountering vulnerabilities.

Lastly, current works do not consider the use environment of IoT devices and various protocols for communication. Although the current works use various methods, such as peripheral modeling, combined with symbolic execution, to improve code coverage of fuzzing, but the peripheral inputs generated by these methods does not have practical significance, and state-based peripheral inputs are difficult to generate by these methods. Only by considering the use environment of the IoT devices and the protocols that firmware interacts with outside world can more meaningful inputs be generated and code coverage can be more effectively improved. Therefore, in future work, one could consider generating inputs efficiently by modeling communication protocol.

5 CONCLUSION

The main objective of this literature review was to summarize the existing IoT fuzzing techniques in order to identify existing software vulnerabilities, state-of-the-art fuzzing techniques, and performance evaluation. Moreover, the main findings obtained from this literature review are summarized below according to the defined research questions.

We divide the existing firmware fuzzing work into real-device-based fuzzing and simulation-based fuzzing according to the firmware execution environment, the problems solved by the two fuzzing methods are different. Real-device-based fuzzing focuses on how to interact with real devices and obtain the status of the device to determine whether a vulnerability is triggered. This type of methods cannot obtain the exact location and category of the vulnerabilities. Simulation-based fuzzing focuses on solving the problem that the simulation platform cannot be compatible with peripheral hardware and generating effective test cases to improve code coverage of fuzzing. Simulation-based fuzzing can obtain richer run-time information to assist vulnerability detection.

Since the current research focus is still on solving the firmware execution environment or interaction methods, the existing work does not regard detecting more types of vulnerabilities as its main goal. In the real-device-based fuzzing, the device execution logs and the run-time states of devices are the main factors to determine whether a vulnerability is triggered. The detected vulnerabilities are mainly memory corruption vulnerabilities or devices-crash-related vulnerabilities. In the simulation-based fuzzing, most tools use AFL [27] as the fuzzer, and regard whether the system crashes as the factor that triggers the memory corruption vulnerabilities, but the simulator can provide richer run-time information. Only FIRMADYNE [9] and Laelaps [7] use this information to detect other types of vulnerabilities.

There is a huge difference between firmware fuzzing and general software fuzzing. The binary files of firmware are difficult to obtain, the test environment is difficult to build, the firmware code is more complicated, and the firmware test cases are difficult to generate. General software fuzzing tools are only AFL [27] partially used in firmware fuzzing. Other tools are difficult to be applied to firmware fuzzing. Due to the complex architecture of the hardware devices where the firmware is located, it is difficult to integrate traditional analysis techniques such as static analysis and symbolic execution into the fuzzing to improve the efficiency of the test.

The current fuzzing technology cannot be applied to commercial IoT firmware on a large scale. For this reason, we propose future research recommendations. A complete IoT firmware test benchmark is conducive to firmware testing research; a complete simulation environment helps researchers focus on solving the problems of the fuzzing itself; full use of the firmware run-time information extracted from the simulator helps to discover more types of vulnerabilities; full consideration of the scenes information to be used and communication protocols information of the IoT devices can help improve the fuzzing efficiency.

ACKNOWLEDGMENTS

This work was supported by the National Natural Science Foundation of China under Grant No.62032010.

REFERENCES

[1] Amazon. 2021. FreeRTOS. https://www.freertos.org/. Accessed: 03/25/2021.

[2] Pedram Amini and Aaron Portnoy. 2013. Sulley: Pure python fully automated and unattended fuzzing framework. https://github.com/OpenRCE/sulley/. Accessed: 03/25/2021.

[3] ARM. 2021. ETM. https://developer.arm.com/documentation/ddi0337/h/embedded-trace-macrocell/about-the-etm. Accessed: 03/25/2021.

[4] Sara Abbaspour Asadollah, Daniel Sundmark, Sigrid Eldh, and Hans Hansson. 2018. A runtime verification tool for detecting concurrency bugs in freertos embedded software. In *2018 17th International Symposium on Parallel and Distributed Computing (ISPDC)*. IEEE, 172–179.

[5] Oleksandr Bazhaniuk, John Loucaides, Lee Rosenbaum, Mark R. Tuttle, and Vincent Zimmer. 2015. Symbolic Execution for BIOS Security. In *9th USENIX Workshop on Offensive Technologies (WOOT 15)*.

[6] Fabrice Bellard. 2021. QEMU. https://www.qemu.org/. Accessed: 03/25/2021.

[7] Chen Cao, Le Guan, Jiang Ming, and Peng Liu. 2020. Device-agnostic Firmware Execution is Possible: A Concolic Execution Approach for Peripheral Emulation. In *Annual Computer Security Applications Conference*. 746–759.

[8] Prakash Chandrasekaran, Shibu Kumar KB, Remish L Minz, Deepak D'Souza, and Lomesh Meshram. 2014. A multi-core version of FreeRTOS verified for datarace and deadlock freedom. In *2014 Twelfth ACM/IEEE Conference on Formal Methods and Models for Codesign (MEMOCODE)*. IEEE, 62–71.

[9] Daming D Chen, Maverick Woo, David Brumley, and Manuel Egele. 2016. Towards Automated Dynamic Analysis for Linux-based Embedded Firmware.. In *NDSS*, Vol. 16. 1–16.

[10] Jiongyi Chen, Wenrui Diao, Qingchuan Zhao, Chaoshun Zuo, Zhiqiang Lin, XiaoFeng Wang, Wing Cheong Lau, Menghan Sun, Ronghai Yang, and Kehuan Zhang. 2018. IoTFuzzer: Discovering Memory Corruptions in IoT Through App-based Fuzzing. In *NDSS*.

[11] Kai Cheng, Qiang Li, Lei Wang, Qian Chen, Yaowen Zheng, Limin Sun, and Zhenkai Liang. 2018. DTaint: detecting the taint-style vulnerability in embedded device firmware. In *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 430–441.

[12] Abraham A Clements, Eric Gustafson, Tobias Scharnowski, Paul Grosen, David Fritz, Christopher Kruegel, Giovanni Vigna, Saurabh Bagchi, and Mathias Payer. 2020. HALucinator: Firmware Re-hosting Through Abstraction Layer Emulation. In *29th USENIX Security Symposium (USENIX Security 20)*. 1201–1218.

[13] Lucian Cojocar, Jonas Zaddach, Roel Verdult, Herbert Bos, Aurélien Francillon, and Davide Balzarotti. 2015. PIE: Parser identification in embedded systems. In *Proceedings of the 31st Annual Computer Security Applications Conference*. 251–260.

[14] Nassim Corteggiani, Giovanni Camurati, and Aurélien Francillon. 2018. Inception: System-wide security testing of real-world embedded systems software. In *27th USENIX Security Symposium (USENIX Security 18)*. 309–326.

[15] Andrei Costin, Jonas Zaddach, Aurélien Francillon, and Davide Balzarotti. 2014. A large-scale analysis of the security of embedded firmwares. In *23rd USENIX Security Symposium (USENIX Security 14)*. 95–110.

[16] Yaniv David, Nimrod Partush, and Eran Yahav. 2018. Firmup: Precise static detection of common vulnerabilities in firmware. *ACM SIGPLAN Notices* 53, 2 (2018), 392–404.

[17] Drew Davidson, Benjamin Moench, Thomas Ristenpart, and Somesh Jha. 2013. FIE on firmware: Finding vulnerabilities in embedded systems using symbolic execution. In *22nd USENIX Security Symposium (USENIX Security 13)*. 463–478.

[18] Michael Eddington. 2011. Peach fuzzing platform. http://www.peachfuzzer.com/products/peach-platform/. Accessed: 03/25/2021.

[19] Bo Feng, Alejandro Mera, and Long Lu. 2020. P2IM: Scalable and Hardware-independent Firmware Testing via Automatic Peripheral Interface Modeling. In *29th USENIX Security Symposium (USENIX Security 20)*.

[20] Qian Feng, Minghua Wang, Mu Zhang, Rundong Zhou, Andrew Henderson, and Heng Yin. 2017. Extracting conditional formulas for cross-platform bug search. In *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*. 346–359.

[21] Qian Feng, Rundong Zhou, Chengcheng Xu, Yao Cheng, Brian Testa, and Heng Yin. 2016. Scalable graph-based bug search for firmware images. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 480–491.

[22] Vijay Ganesh, Tim Leek, and Martin Rinard. 2009. Taint-based directed whitebox fuzzing. In *2009 IEEE 31st International Conference on Software Engineering*. IEEE, 474–484.

[23] Jian Gao, Xin Yang, Ying Fu, Yu Jiang, Heyuan Shi, and Jiaguang Sun. 2018. Vulseeker-pro: Enhanced semantic learning based binary vulnerability seeker with emulation. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 803–808.

[24] Jian Gao, Xin Yang, Ying Fu, Yu Jiang, and Jiaguang Sun. 2018. Vulseeker: a semantic learning based vulnerability seeker for cross-platform binary. In *2018 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 896–899.

[25] github.com. 2021. github-2018-03-01-ddos-incident-report. https://github.blog/2018-03-01-ddos-incident-report/. Accessed: 03/25/2021.

[26] Google. 2019. Syzkaller. https://github.com/google/syzkaller/. Accessed: 03/25/2021.

[27] Google. 2021. American Fuzzy Lop. http://lcamtuf.coredump.cx/afl/. Accessed: 03/25/2021.

[28] Serge Gorbunov and Arnold Rosenbloom. 2010. Autofuzz: Automated network protocol fuzzing framework. *IJCSNS* 10, 8 (2010), 239–245.

[29] Ivan Gotovchits, Rijnard Van Tonder, and David Brumley. 2018. Saluki: finding taint-style vulnerabilities with static property checking. In *Proceedings of the NDSS Workshop on Binary Analysis Research*.

[30] Grant Hernandez, Farhaan Fowze, Dave Tian, Tuba Yavuz, and Kevin RB Butler. 2017. Firmusb: Vetting usb device firmware using domain informed symbolic execution. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2245–2262.

[31] Seyedrebvar Hosseini, Burak Turhan, and Dimuthu Gunarathna. 2017. A systematic literature review and meta-analysis on cross project defect prediction. *IEEE Transactions on Software Engineering* 45, 2 (2017), 111–147.

[32] Intel. 2021. IntelPT. https://software.intel.com/content/www/us/en/develop/blogs/processor-tracing.html. Accessed: 03/25/2021.

[33] Staffs Keele et al. 2007. *Guidelines for performing systematic literature reviews in software engineering*. Technical Report. Technical report, Ver. 2.3 EBSE Technical Report. EBSE.

[34] Mingeun Kim, Dongkwan Kim, Eunsoo Kim, Suryeon Kim, Yeongjin Jang, and Yongdae Kim. 2020. FirmAE: Towards Large-Scale Emulation of IoT Firmware for Dynamic Analysis. In *Annual Computer Security Applications Conference*. 733–745.

[35] Karl Koscher, Tadayoshi Kohno, and David Molnar. 2015. SURROGATES: Enabling near-real-time dynamic analyses of embedded systems. In *9th USENIX Workshop on Offensive Technologies (WOOT 15)*.

[36] MIT Lincoln Laboratory, NYU, and Northeastern University. 2021. PANDA. https://github.com/panda-re/panda/. Accessed: 03/25/2021.

[37] Jun Li, Bodong Zhao, and Chao Zhang. 2018. Fuzzing: a survey. *Cybersecurity* 1, 1 (2018), 1–13.

[38] Zheng Lu, Christopher Steinmuller, and Supratik Mukhopadhyay. 2013. Towards formal verification of a commercial wireless router firmware. In *2013 IEEE 37th Annual Computer Software and Applications Conference*. IEEE, 639–647.

[39] Alejandro Mera, Bo Feng, Long Lu, Engin Kirda, and William Robertson. 2020. DICE: Automatic Emulation of DMA Input Channels for Dynamic Firmware Analysis. *arXiv preprint arXiv:2007.01502* (2020).

[40] Marius Muench, Dario Nisi, Aurélien Francillon, and Davide Balzarotti. 2018. Avatar2: A multi-target orchestration platform. In *Proc. Workshop Binary Anal. Res.(Colocated NDSS Symp.)*, Vol. 18. 1–11.

[41] Marius Muench, Jan Stijohann, Frank Kargl, Aurélien Francillon, and Davide Balzarotti. 2018. What You Corrupt Is Not What You Crash: Challenges in Fuzzing Embedded Devices.. In *NDSS*.

[42] Van-Thuan Pham, Marcel Böhme, and Abhik Roychoudhury. 2016. Model-based whitebox fuzzing for program binaries. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*. 543–553.

[43] Chang Qing, Liu Zhongjin, Wang Mengtao, Chen Yu, Shi Zhiqiang, and Sun Limin. 2016. Vdns: an algorithm for cross-platform vulnerability searching in binary firmware. *Journal of Computer Research and Development* 53, 10 (2016), 2288.

[44] Nilo Redini, Aravind Machiry, Dipanjan Das, Yanick Fratantonio, Antonio Bianchi, Eric Gustafson, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. 2017. Bootstomp: on the security of bootloaders in mobile devices. In *26th USENIX Security Symposium (USENIX Security 17)*. 781–798.

[45] Nilo Redini, Aravind Machiry, Ruoyu Wang, Chad Spensky, Andrea Continella, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. 2020. Karonte: Detecting insecure multi-binary interactions in embedded firmware. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1544–1561.

[46] WIND RIVER. 2021. Simics. https://www.windriver.com/products/simics/. Accessed: 03/25/2021.

[47] Segger. 2021. J-Trace. https://www.segger.com/products/debug-probes/j-trace/. Accessed: 03/25/2021.

[48] Yan Shoshitaishvili, Ruoyu Wang, Christophe Hauser, Christopher Kruegel, and Giovanni Vigna. 2015. Firmalice-Automatic Detection of Authentication Bypass Vulnerabilities in Binary Firmware.. In *NDSS*, Vol. 1. 1–1.

[49] Robert Swiecki. 2017. Honggfuzz: A general-purpose, easy-to-use fuzzer with interesting analysis options. *URI: https://github.com/google/honggfuzz (visited on 03/25/2021)* (2017).

[50] Chin-Wei Tien, Tsung-Ta Tsai, Yi Chen, and Sy-Yen Kuo. 2018. UFO-Hidden Backdoor Discovery and Security Verification in IoT Device Firmware. In *2018 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 18–23.

[51] Yu Wang, Junjing Shi, Linzhang Wang, Jianhua Zhao, and Xuandong Li. 2015. Detecting Data Races in Interrupt-Driven Programs Based on Static Analysis and

Dynamic Simulation. In *Proceedings of the 7th Asia-Pacific Symposium on Internetware* (Wuhan, China) *(Internetware '15)*. Association for Computing Machinery, New York, NY, USA, 199–202. https://doi.org/10.1145/2875913.2875943

[52] Yu Wang, Linzhang Wang, Tingting Yu, Jianhua Zhao, and Xuandong Li. 2017. Automatic Detection and Validation of Race Conditions in Interrupt-Driven Embedded Software. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Santa Barbara, CA, USA) *(ISSTA 2017)*. Association for Computing Machinery, New York, NY, USA, 113–124. https://doi.org/10.1145/3092703.3092724

[53] Z. Wang, Y. Zhang, and Qixu Liu. 2013. RPFuzzer: A Framework for Discovering Router Protocols Vulnerabilities Based on Fuzzing. *KSII Trans. Internet Inf. Syst.* 7 (2013), 1989–2009.

[54] Xiaojun Xu, Chang Liu, Qian Feng, Heng Yin, Le Song, and Dawn Song. 2017. Neural network-based graph embedding for cross-platform binary code similarity detection. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 363–376.

[55] Chen Yu, Liu Zhongjin, Zhao Weiwei, Ma Yuan, Shi Zhiqiang, and Sun Limin. 2018. A Large-Scale Cross-Platform Homologous Binary Retrieval Method. *Journal of Computer Research and Development* 55, 7 (2018), 1498.

[56] Jonas Zaddach, Luca Bruno, Aurelien Francillon, Davide Balzarotti, et al. 2014. AVATAR: A Framework to Support Dynamic Security Analysis of Embedded Systems' Firmwares.. In *NDSS*, Vol. 14. 1–16.

[57] Yaowen Zheng, Ali Davanian, Heng Yin, Chengyu Song, Hongsong Zhu, and Limin Sun. 2019. FIRM-AFL: high-throughput greybox fuzzing of iot firmware via augmented process emulation. In *28th USENIX Security Symposium (USENIX Security 19)*. 1099–1114.

[58] Y Zou, S Liu, N Yu, et al. 2018. IoT device recognition framework based on Web search. *J. Cyber Secur* 3, 4 (2018), 25–40.

A OVERVIEW OF FIRMWARE FUZZING TOOLS

Here is a detailed list of fuzzing tools and their execution environment, support architectures, support systems, target peripherals or functions, and types of vulnerabilities that can be detected in the Table 1.

Table 1: Firmware Fuzzing Tools List

Fuzzing Tool	Execution Environment	Support Architecture	Support System	Target Peripheral/Function	Target Vulnerabilities
RPFuzzer[53]	real device	unlimited	unlimited	router protocol	DoS vulnerabilities, router reboot, zombie process
Avatar[56]	QEMU and real device	ARM	unlimited	unlimited	not support
SURROGATES[35]	QEMU and real device	ARM	unlimited	unlimited	not support
Firmadyne[9]	QEMU	ARMel, MIPSel, MIPSeb	Linux	network service	command injection, buffer overflow, information disclosure, sercomm configuration dump, MiniUPnPd Denial of Service, OpenSSL ChangeCipherSpec
IOTFuzzer[10]	real device	unlimited	unlimited	communication interface	memory corruption vulnerabilities
Avatar2[40]	QEMU/PANDA	ARM	unlimited	unlimited	not support
Firm-AFL[57]	QEMU	ARMel, MIPSel, MIPSeb	Linux	network service	crashes
FirmAE[34]	QEMU	ARMel, MIPSel, MIPSeb	Linux	web service	memory corruption vulnerabilities, information leak, command injection, password disclosure, authentication bypass
P2IM[19]	QEMU	ARM, MIPS, AVR, RISC-V	unlimited	unlimited	crashes/hangs
DICE[39]	QEMU/PIC32	ARM, MIPS	unlimited	DMA input channels	crashes/hangs
HALucinator[12]	QEMU	ARM	unlimited	unlimited	crashes
Lealaps[7]	QEMU	ARM	unlimited	unlimited	memory corruption vulnerabilities